

Case Mibo Smart

Aplicativo Kotlin Multiplatform para casa inteligente
Android · iOS · API do parceiro



Quatro ondas, e uma ordem de corte decidida antes

O risco de um case não é escrever código: é descobrir tarde que o caminho crítico não cabia no prazo. Então o corte foi combinado no começo, não negociado no fim.

Onda	Escopo	Por que nesta ordem
0	<i>Walking skeleton</i> : repositório, CI, esqueleto de módulos, SPEC, ADRs, testes de arquitetura	a trilha começa antes do código; CI vermelho no dia 1 custa horas, no dia 4 custa a entrega
1	Sessão e token: tela, validação, expiração	primeira fatia vertical de verdade; ela constrói o cliente HTTP e o cofre <i>por necessidade própria</i> , não como "fatia de fundação"
2	Em paralelo: lista + filtro + paginação · vídeo ao vivo · fechadura	as três não dependem entre si; vídeo e fechadura são o "hardware real", onde estão as surpresas
3	Histórico, renovação de token, iOS, módulo Java, R8, este documento	só depois de o núcleo funcionar no Android

CIRCUIT BREAKER, ESCRITO ANTES DE PRECISAR

Se o prazo apertasse, a onda 3 encolheria

NESTA ORDEM

Case Mibo Smart

: módulo Java → histórico → fallback do vídeo → R8. Vídeo e fechadura nunca foram cortáveis, porque são o coração do enunciado.

2

Uma fatia é vertical, e cabe numa revisão

Fatia horizontal ("agora a camada de dados") entrega risco acumulado. Fatia vertical atravessa `domain` → `data` → `app` → `tela` e entrega algo observável.

- **Teto de ~400 linhas executáveis de produção** por PR, medido por script (`tools/executable-lines.py`), não pela mão.
- **Testes são contados e não entram no teto.** Quem os limita são os critérios de aceite, não um número.
- **Uma issue por fatia**, com critérios EARS e os arquivos que ela toca.
- **Um PR por issue**, com o *porquê* no corpo, em português, para quem revisa.

O CICLO, DO ENUNCIADO AO COMMIT

SPEC (50 critérios EARS)

- ADR quando a decisão é estrutural
- Issue por fatia vertical
- PR com evidência real de teste
- merge por humano, sempre

Nenhuma etapa é cerimônia: cada uma existe porque a anterior produziria retrabalho sem ela.

Cada dependência precisou justificar a própria existência

Regra do projeto: nenhuma dependência entra para "fazer compilar". Ela entra com uma linha de ADR dizendo o que resolve e o que foi recusado no lugar.

Escolha	Por quê	Alternativa recusada
Kotlin Multiplatform 2.4 Compose Multiplatform 1.11	lógica e UI compartilhadas de verdade; o iOS consome o mesmo framework	dois apps nativos, que duplicariam as regras de negócio, que é onde estão os erros
Ktor 3.5	cliente multiplataforma, <code>MockEngine</code> para testar contrato sem rede	Retrofit, que só roda na JVM
Koin 4.2	DI em runtime, funciona em Kotlin/Native sem processador de anotação	Hilt/Dagger, que não atravessa para iOS
SQLDelight 2.3	SQL tipado e multiplataforma; o cache da lista existe por causa do orçamento de requisições	Room, que só roda no Android
Media3 no Android <code>WKWebView</code> no iOS	o parceiro entrega fMP4 sobre HTTPS; <code>expect/actual</code> só na superfície do player	VLCKit no iOS, dezenas de MB para um caso que a página do parceiro já resolve
Konture · Roborazzi · Kover · Mokkery · Turbine Case Mibo Smart	arquitetura como teste que falha o build ; goldens de tela; cobertura medida; mocks em Native	revisão manual de fronteira, que erode em uma sprint

Dependências apontam só para dentro

Quatro módulos e uma regra. O que o compilador não garante, o teste de arquitetura garante. E ele falha o build, não emite aviso.

:shared:domain

Kotlin puro. Modelos, contratos de repositório, erros tipados, resultados selados. **Nada de Ktor, Koin, SQLDelight, Android ou Compose.**

:shared:data

O parceiro mora aqui, e só aqui: cliente HTTP, DTOs com `@SerialName`, mapeadores, cache, cofre do token. As esquisitices do contrato morrem nesta fronteira.

:shared:app

Casos de uso, ViewModels e telas Compose, por feature. Gera o framework `Shared` do iOS.

TRÊS DECISÕES QUE SUSTENTAM O RESTO

Erro é valor.

A camada de dados lança exceções tipadas; cada caso de uso devolve o resultado da *sua* intenção. O `when` da tela é exaustivo e o compilador cobra.

Um estado imutável por tela.

Estados de hardware são tipos selados, não booleanos combinados.

COMO PODERIA SER APRIMORADA

- MODULARIZAR POR FEATURE

, não por camada, quando o time crescer: hoje `:shared:app` concentra quatro features e é o arquivo mais disputado num merge.

- UM SEGUNDO PARCEIRO DE VERDADE

validaria o ADR-004. Hoje a fronteira é boa por construção, não por prova.

O repositório conta a história, não só o resultado

Quem abre só o `main` vê um terço do trabalho. A rastreabilidade requisito → fatia → PR → commit é o produto tanto quanto o app.

- **5 épicos:** Sessão, Dispositivos, Vídeo, Fechadura, Fundação/entrega.
- **Milestones Wave 0–3** e um Project com campo `Wave`: progresso por onda, sem abrir issue por issue.
- **Labels** `type:*`, `rf:RF0x`, `area:*`. O mapeamento requisito → fatia sai dos rótulos, não da memória de quem escreveu o README.
- **Um PR por issue**, com `Closes #n`, corpo em português, critérios marcados só quando provados, e saída real de teste colada.
- **Squash-merge por humano**, sempre. Nenhum merge automático.

ONDE ESTÁ CADA COISA

`docs/specs/SPEC.md` · critérios EARS
`docs/adr/` · 27 decisões, com as opções recusadas
`docs/api-contract.md` · o contrato como observado
`AI-LOG.md` · uso de IA, incluindo os erros
`docs/PROCESS.md` · como fatia, PR e merge funcionam
`docs/guides/` · token, Swagger, build, demonstração

CI EM TRÊS ESTÁGIOS

verify

· arquitetura, testes JVM e capturas de tela, em todo PR

android

· APK e lint

ios

· macOS: testes no simulador, framework, `xcodebuild`

O contrato da API moldou o código

Oito contradições documentadas. Nenhuma virou um `if` espalhado: cada uma tem um lugar só e um teste que falha se esse lugar mudar.

O que o contrato faz	O que o código faz a respeito
HTTP é sempre <code>200</code> ; o Swagger documenta 402/404/500 como código HTTP	<code>EnvelopeReader</code> lê o status do corpo
Dois formatos de envelope para a mesma API	o mesmo leitor aceita os dois; nenhuma tela sabe disso
Token recusado é indistinguível de erro genérico, exceto pelo texto	regra de negócio no caso de uso, não no cliente HTTP
A fechadura só responde por namespace composto <code><fechadura>_<central>_<idProdutoPai></code>	um tipo <code>LockAddress</code> ; a montagem fica num lugar só
<code>volume/v1</code> exige <code>productId</code> mas a propriedade é <code>idProduto</code>	vão os dois campos, e a quarentena fica no DTO
O mesmo endpoint responde diferente em dois hosts , os dois com <code>200</code>	streaming vai para o portal, o resto para a API; um teste afirma o host de cada requisição

O orçamento de requisições é o eixo do desenho

A conta de teste tem franquia finita e cada chamada conta, inclusive validar o token. Isso deixou de ser restrição e virou princípio de projeto.

- **Cache da lista:** a tela abre com o que já sabe, antes de a rede responder.
- **Zero polling**, em lugar nenhum do app.
- **Classificação por `modelo`** antes de chamar `funcoes`. O resultado fica em cache permanente por câmera.
- **Validação local do formato do token:** colagem truncada falha no campo, sem gastar requisição.
- **Trocar o chip de filtro não custa nada** quando a lista completa já está em memória.
- **Uma leitura de confirmação** na fechadura, com ação do usuário, em vez de polling.

E A RESTRIÇÃO MORDEU

Na véspera da entrega a franquia da conta chegou a zero, e a API passou a responder `401 "verifique os seus limites disponíveis"` em todos os endpoints.

O desenho já estava pronto para isso, o que mostra que o orçamento não era preocupação teórica.

Um comando é um estado *sobre* a fechadura

A tela mais difícil do app não é a que mostra vídeo. É a que precisa não mentir sobre uma porta.

O ESTADO QUE O APP SE RECUSA A SIMPLIFICAR

Comando enviado, porta não respondeu

Não é "a porta recusou". Não é "a porta abriu". É
NÃO SEI

, e ninguém consegue distinguir as duas coisas a partir da API.

Fingir que o app distingue seria o erro mais caro desta tela, porque o usuário está em frente à porta.

- **Nenhuma tela afirma que a porta abriu** antes de uma leitura concordar com o comando.
- **Offline e "abertura remota desabilitada" não são estados** da tela: são leituras *sobre* a fechadura. A tela mostra porta, volume e pré-condição juntos.
- **Escrita é opt-in no build.** `mudar-volume` e `habilitar-abrir-remoto` terminam em hardware de uma conta compartilhada, então o padrão é **desligado**.
- **Uma leitura secundária não veta a tela:** o endpoint de volume responde 500 em cinco das seis fechaduras da conta, e o estado da porta, que leu certo, continua aparecendo.

Qualidade: o que barra o merge

Cobertura é medida e publicada. O que **impede** um merge é outra coisa.

TESTES

314

em 5 módulos

ADRS

27

com opções recusadas

PULL REQUESTS

55

um por fatia

COBERTURA

61,5%

linhas · 61% ramos

BARRA O MERGE

Testes de arquitetura

- dez regras de fronteira, como teste JVM que falha o build.

A suíte

- regras de negócio no host, contrato com `MockEngine`, nenhum teste toca a API real.

A guarda de deriva

- acrescentar um estado de tela sem acrescentar a captura quebra o build.

NÃO BARRA, E É PROPOSITAL

Cobertura.

Ela exclui `*.platform*` e a UI Compose, provadas por preview e golden; contá-las inflaria o denominador com código que nenhum teste unitário deveria tocar.

Está abaixo da meta de 80% que o time mencionou. O caminho até lá é conhecido: as telas têm 42 goldens que hoje são evidência de revisão e ainda não são porta de regressão.

Uso de IA, com guarda-corpos

A IA acelera escrita de código e erode fronteira na mesma velocidade. O projeto foi montado para que a segunda parte não passe.

O que a barra

Dez regras de arquitetura como teste JVM que **falha o build**. Guarda de deriva nas telas. Nenhum merge automático: squash por humano, sempre.

O que fica registrado

`AI-LOG.md`, com 53 entradas. Cada decisão estrutural tem ADR com as opções recusadas, então o *porquê* sobrevive a quem escreveu.

A regra de trabalho

Premissa escrita em ticket é **hipótese até ser medida**. Medir antes de implementar custa minutos; o contrário custa a mudança inteira.

O QUE ISSO PRODUZIU NA PRÁTICA

Decisões contra a corrente do "mais fácil": polling recusado na fechadura por causa do orçamento de requisições; o gerador de previews do Roborazzi recusado depois de medir que não funciona neste layout KMP; uma otimização inteira descartada depois de medir que não economizava nada.

O que eu faria em seguida, em ordem

Separado por quem depende de quem: o que é meu, o que é do parceiro, e o que é decisão de produto.

Técnico, meu

Escada de confirmação da fechadura: dar tempo ao hardware antes de declarar não confirmado.

Goldens como porta de regressão, com baseline gravada no CI.

Ordenação escolhível da lista, por tipo, nome ou status.

Afinar os números do vídeo contra a câmera real. Hoje seguem marcados como suposição.

Depende do parceiro

Vídeo ao vivo. A sessão é criada normalmente e o transcodificador encerra o fluxo em 15 s sem enviar dados. Reportado com as medições.

O endpoint de volume da fechadura responde 500 em cinco das seis fechaduras.

Dois hosts com o mesmo endpoint respondendo formas diferentes, ambos com 200.

Produto

Da pesquisa com 270 avaliações reais, fora do escopo deste case:

Gravações, timeline e cartão SD: maior cluster de reclamação depois de vídeo.

Notificação de dispositivo com granularidade: o que incomoda é push de marketing; o que pedem é alerta de segurança.

Um app e uma conta para todas as linhas.

E um achado que não é backlog

Lendo as avaliações do app oficial, um relato de 06/03/2026 descreve uma senha excluída na fechadura que continuava abrindo a porta, e um histórico que não identifica qual senha foi usada.

POR QUE ISTO NÃO VIROU UM CARD DE FEATURE

Se procede, é revogação de credencial que não chega ao hardware. Isso pede
TRATAMENTO DE INCIDENTE

: reproduzir, avaliar alcance, corrigir e comunicar. Não uma linha no backlog atrás de outras dez.

Está classificado assim na pesquisa desde o primeiro dia, e é o motivo de eu ter lido avaliações antes de escrever a primeira issue:

O QUE OS USUÁRIOS ESCREVEM CONTÉM COISAS QUE NENHUMA ESPECIFICAÇÃO CONTÉM.

Obrigado

Repositório, decisões e o log de uso de IA:

github.com/rpauloj/case-mibo-smart

